Linux for DevOps

A Beginner-Friendly Textbook with Real Analogies

Kubernetes | Shell Scripting | Networking | Permissions

What's Inside

1. Linux Basics

What is Linux?

Linux is a free, open-source operating system kernel created by Linus Torvalds in 1991. It powers web servers, smartphones, supercomputers, and Kubernetes clusters worldwide.

> **Easy Analogy:** Think of an OS like a restaurant. The Linux kernel is the kitchen that does the actual cooking (managing CPU, RAM, disk). The waiter, tables, and menu are the shell & tools built on top. You interact with the front-end but the kitchen runs everything.

Kernel vs Operating System

Kernel (Engine)	Operating System (Full Car)
Manages hardware: CPU, RAM, Disk	Kernel + Shell + GUI + Package Manager
Core program -- loads first at boot	Everything you interact with daily
Like a car engine -- powerful, raw	Like a full car -- engine, doors, GPS, AC

Popular Linux Distributions

A distribution (distro) = Linux kernel + pre-installed tools + package manager. Like different flavors of the same ice cream base.

Distro	Best For	Package Manager
Ubuntu	Beginners, cloud, Docker, DevOps labs	`apt`
CentOS	Servers, stable enterprise environments	`yum / dnf`
Red Hat (RHEL)	Enterprise production, paid support	`yum / dnf`

2. File System Structure (Very Important)

In Linux, EVERYTHING is a file -- even hardware devices like your hard drive (/dev/sda) and running processes (/proc/1234). The file system is a single tree starting from the root /.

Easy Analogy: The Linux file system is like a city. The root / is City Hall -- everything branches from there. /etc is the Rules & Regulations building. /home is the Residential Area. /var/log is the Police Records Office. /tmp is a public bulletin board that gets wiped every night.

The Directory Tree

```
/   (Root -- The Boss of Everything)
|-- /bin       -> Essential commands (ls, cp, mv ...)
|-- /etc       -> Config files   (like your Settings app)
|-- /home      -> User folders   (like C:\Users\ on Windows)
|-- /root      -> Root (admin) user's own home folder
|-- /var
|    |-- /var/log   -> System & application logs
|    '-- /var/run   -> Runtime data (PID files, sockets)
|-- /usr       -> Programs, libraries, documentation
|-- /tmp       -> Temporary files (auto-deleted on reboot)
|-- /opt       -> Optional 3rd-party software (Jenkins etc.)
'-- /proc      -> Virtual FS: live process & hardware info
```

Important Directories at a Glance

Directory	Purpose & DevOps Use
`/etc`	Config files for system & apps. Edit nginx.conf, sshd_config here.
`/var/log`	All log files. Check here when pods or services crash.
`/home`	User home directories. Store your scripts and projects here.
`/root`	The root (admin) user's home. Admin-only area.
`/proc`	Virtual FS with live process/hardware data. Read-only virtual files.
`/tmp`	Temporary files. Cleared every reboot. Never store important data here.
`/opt`	Optional 3rd-party software. Install Jenkins, Kubernetes tools here.
`/usr/bin`	Most user-facing commands: grep, curl, python, git...

Kubernetes Tip: When a K8s pod crashes, first check /var/log on the node. Then use: kubectl logs <pod-name> --previous

3. File & Directory Commands

These are your daily-driver commands. Master these and you can navigate any Linux server confidently.

Easy Analogy: Commands are like verbs in English. pwd = 'Where am I?' ls = 'What is here?' cd = 'Take me to...' mkdir = 'Build a new room' rm -rf = 'Demolish the whole building now' cat = 'Read this document aloud'

Command	Use / Description
`pwd`	Print current directory path. 'Where am I right now?'
`ls -la`	List ALL files including hidden, with permissions, size & date
`cd /etc`	Change directory -- move into /etc folder
`mkdir myapp`	Create a new directory named myapp
`rm -rf folder/`	Delete folder and ALL its contents permanently. No undo!
`cp file.txt /tmp/`	Copy file.txt to the /tmp directory
`mv old.txt new.txt`	Rename file (or move it to another location)
`touch app.sh`	Create an empty file named app.sh
`cat /etc/hosts`	Display full content of a file in the terminal
`less bigfile.log`	View large file page by page (press q to quit)
`grep 'error' app.log`	Search for the word 'error' inside app.log
`find / -name '*.conf'`	Find all files ending in .conf starting from root

Quick Examples

```
# Where am I right now?
pwd

# List files with full details
ls -la /var/log

# Create project folder and file in one line
mkdir myproject && touch myproject/deploy.sh

# Backup nginx config before editing
cp /etc/nginx/nginx.conf /tmp/nginx.conf.bak

# Search logs for errors (last 20 lines only)
grep 'ERROR' /var/log/syslog | tail -20
```

4. File Permissions (Interview Favorite)

Every file in Linux has permissions. This controls who can read, write, or execute it. Critical for Kubernetes volume mounts, SSH keys, and security.

Easy Analogy: Permissions are like a building's key cards. Owner (you) has master key. Group (your team) has limited access card. Others (everyone else) can only look through the glass window. chmod is the security manager who grants or revokes access cards.

Permission Breakdown

Symbol	Owner (u) rwx	Group (g) r-x	Others (o) r--
-rwxr-xr--	rwx = 7	r-x = 5	r-- = 4

The Octal Number System

r = read = 4 | w = write = 2 | x = execute = 1 Add them up: rwx = 4+2+1 = 7 r-x = 4+0+1 = 5 r-- = 4+0+0 = 4

chmod	Permissions	Meaning
chmod 777	rwxrwxrwx	Everyone can do everything (insecure -- avoid in production)
chmod 755	rwxr-xr-x	Owner full; others read+execute (common for scripts)
chmod 644	rw-r--r--	Owner read/write; others read only (common for config files)
chmod 600	rw-------	Only owner can read/write (SSH keys must be 600!)
chmod 400	r--------	Read-only even for owner (certs, secret config)

Change Permissions & Ownership

```
# Give owner full, others read+execute (common for scripts)
chmod 755 deploy.sh

# Change file owner to user 'devops', group 'team'
chown devops:team app.conf

# Change owner recursively for entire directory
chown -R devops:team /opt/myapp

# Make a script executable and run it
chmod +x run.sh && ./run.sh
```

```
# SSH keys MUST be 600 -- if too open, SSH refuses to use them
chmod 600 ~/.ssh/id_rsa
```

K8s Tip: If a pod fails with 'permission denied' on a mounted volume, fix with chmod 755 or chown on the volume path on the host node before mounting.

5. User Management

Linux is a multi-user system. Managing users is essential when setting up servers, CI/CD runners, or Kubernetes nodes.

Easy Analogy: Users in Linux are like employees in a company. Each has an ID badge (UID), belongs to departments (groups), and has specific access rights. The root user is the CEO -- can do anything, anytime, on anything. Use wisely!

Command	Use / Description
`useradd devuser`	Create a new user named devuser
`passwd devuser`	Set or change password for devuser
`usermod -aG sudo devuser`	Add devuser to the sudo (admin) group
`id devuser`	Show UID, GID, and all groups for devuser
`whoami`	Print the currently logged-in username
`su - devuser`	Switch to devuser account (full login shell)
`userdel -r devuser`	Delete user AND their home directory
`cat /etc/passwd`	View all system users (one per line)
`groups devuser`	List all groups that devuser belongs to

```
# Full workflow: create a DevOps user
useradd devops
passwd devops
usermod -aG sudo devops

# Verify the user was created
id devops
# Output: uid=1001(devops) gid=1001(devops) groups=1001(devops),27(sudo)

# Create a non-login service user (for apps like jenkins)
useradd -r -s /bin/false jenkins
```

Best Practice DevOps: Always create dedicated service users (jenkins, k8s, nginx) instead of running services as root. If a service gets hacked, the attacker only gets limited access -- not full root.

6. Process Management (K8s Debugging)

A process is any program currently running. Linux gives you full visibility and control -- just like Kubernetes manages the lifecycle of containers.

Easy Analogy: A process is like a factory worker. ps -ef is taking attendance -- who's working and what are they doing. top/htop is watching the live CCTV to see who's slacking or overworking. kill -9 is like instantly terminating a worker -- no notice, no cleanup.

Command	Use / Description
`ps -ef`	Show ALL running processes with PID, user, and command
`ps aux | grep nginx`	Find the nginx process specifically
`top`	Live dashboard: CPU & memory per process (q to quit)
`htop`	Advanced top with colors and mouse support
`kill -9 PID`	Force-kill a process immediately (no cleanup)
`kill -15 PID`	Graceful shutdown -- let the process clean up first
`systemctl status nginx`	Check if the nginx service is running
`systemctl start nginx`	Start the nginx service
`systemctl stop nginx`	Stop the nginx service
`systemctl restart nginx`	Restart the nginx service
`journalctl -u nginx`	View systemd logs for the nginx service

Find & Kill a Stuck Process

```
# Step 1: Find the process ID (PID)
ps -ef | grep myapp
# Output: devops  1234  1  0 10:00 ?  00:00:01 /usr/bin/myapp
#                 ^^^^  This is the PID

# Step 2: Kill it (force)
kill -9 1234

# OR: kill all processes matching a name
pkill -f myapp

# Check service log when it fails
journalctl -u nginx --since '10 min ago'
```

K8s Connection: When a Kubernetes pod is in CrashLoopBackOff it's like a process that keeps dying. Use kubectl describe pod <name> to see why -- just like systemctl status shows why a service failed.

7. Disk & Memory Monitoring

Running out of disk or memory is one of the top causes of pod crashes and node failures in Kubernetes. Always monitor resources proactively.

Easy Analogy: Disk space is like your house's storage room. df -h shows how full each room is. du -sh shows which furniture takes the most space. free -m is like checking your bank account (RAM) to see how much money (memory) is left before you go bankrupt (OOM kill).

Command	Use / Description
`df -h`	Show disk space for all mounted file systems (human-readable)
`df -h /`	Check how full the root partition specifically is
`du -sh *`	Show size of every file/folder in the current directory
`du -sh /var/log/*`	Find which log files are consuming disk space
`free -m`	Show RAM usage in megabytes (total, used, free, cache)
`free -h`	RAM usage in human-readable format (GB/MB)
`vmstat 1 5`	CPU/memory/disk stats every 1 second, repeated 5 times
`lsblk`	List all disk partitions and their sizes

Real Scenario: Disk Full -- Pods Crashing

```
# Check overall disk usage
df -h
# Filesystem   Size  Used Avail Use% Mounted on
# /dev/sda1     50G   49G  100M  99% /
# ^ Root is 99% full! Pods will fail to write and crash!

# Find the top 10 space consumers
du -sh /var/log/* | sort -rh | head -10

# Safe way to empty a log file (don't delete -- truncate)
truncate -s 0 /var/log/syslog

# Check RAM -- is OOM killer killing your pods?
free -h
dmesg | grep -i 'killed process'
```

K8s Alert: A Kubernetes node with >85% disk usage will start evicting pods automatically. Always keep an eye on df -h on your worker nodes. Set up alerts in Prometheus for this!

8. Networking Commands (Very Important for K8s)

Networking is the backbone of Kubernetes. Services, Ingress, DNS -- all rely on networking fundamentals. These commands help debug connectivity issues.

Easy Analogy: Network = Road system. ip a shows your vehicle's address (IP). ping is honking to see if someone responds. netstat/ss shows which shops (ports) are open for business. curl sends a formal letter to a shop and reads the reply. traceroute maps every road you take to get there.

Command	Use / Description
`ip a`	Show all network interfaces and their IP addresses
`ip route`	Show routing table -- where network traffic is sent
`ping google.com`	Test if a host is reachable (Ctrl+C to stop)
`ping -c 4 10.0.0.1`	Ping exactly 4 times then stop
`netstat -tulnp`	Show all open ports and which process owns them
`ss -tulnp`	Modern faster replacement for netstat
`curl http://localhost:8080`	Send HTTP GET request, print response body
`curl -I http://site.com`	Check HTTP headers and status code only
`wget http://example.com/f.tar.gz`	Download a file from the internet
`nslookup myservice.svc`	DNS lookup -- resolve hostname to IP
`dig myservice.svc`	Advanced DNS lookup with full details
`traceroute 8.8.8.8`	Show every network hop to a destination
`nc -zv host 443`	Test if a specific port is reachable (netcat)

Kubernetes Service Debugging Flow

```
# 1. Check if the pod is running
kubectl get pods

# 2. Check which port the service exposes
kubectl get svc myservice

# 3. Port-forward to test from your laptop
kubectl port-forward svc/myservice 8080:80

# 4. Test the endpoint
curl http://localhost:8080/health
```

```
# 5. Check listening ports inside the pod
kubectl exec -it <pod-name> -- ss -tulnp

# 6. Test internal DNS resolution inside pod
kubectl exec -it <pod-name> -- nslookup
myservice.default.svc.cluster.local
```

K8s Tip: Pods use internal DNS: <service>.<namespace>.svc.cluster.local. Use curl or nslookup INSIDE a pod to test service-to-service communication within the cluster.

9. Package Management

Package managers let you install, update, and remove software. Think of them as the App Store for Linux -- but from the terminal, with full control.

Easy Analogy: apt / yum are like Swiggy for software. You say 'I want nginx' and the package manager finds the right restaurant (repository), downloads the food (software), includes all the side dishes (dependencies), and delivers it ready to eat (ready to run). No manual installation needed!

Ubuntu / Debian -> apt	CentOS / RHEL -> yum
`apt update`	`yum update`
`apt install nginx`	`yum install httpd`
`apt remove nginx`	`yum remove httpd`
`apt upgrade`	`yum upgrade`
`apt search docker`	`yum search docker`

Pro Tip : Always run 'apt update' BEFORE 'apt install'. Otherwise you might install an outdated version from a stale cache. Think of it as refreshing the Swiggy menu before ordering.

Useful Package Commands

```
# Ubuntu/Debian: Install Docker
apt update && apt install docker.io -y

# CentOS/RHEL: Install Docker
yum install docker -y
systemctl enable --now docker

# Check if a package is installed
dpkg -l | grep nginx          # Ubuntu
rpm -qa | grep nginx          # CentOS

# See what files a package installed
dpkg -L nginx
```

10. Shell Scripting Basics

Shell scripting lets you automate repetitive tasks. Instead of running 10 commands manually every day, write them once in a script and execute with a single command.

Easy Analogy: A shell script is like a step-by-step recipe card. Instead of remembering every cooking step, you write it once on a card. Next time, just follow the card. In DevOps: scripts deploy apps, rotate logs, health-check services, and set up new servers automatically.

Script Structure & Core Concepts

```
#!/bin/bash
# ^ Shebang: tells Linux to use bash for this script

# --- Variables ---
APP_NAME="myapp"
PORT=8080
echo "Starting $APP_NAME on port $PORT"

# --- If / Else ---
if [ -f "/etc/nginx/nginx.conf" ]; then
  echo "Config found!"
else
  echo "Config missing! Creating..."
fi

# --- For Loop ---
for service in nginx docker kubelet; do
  systemctl status $service
done

# --- Function ---
check_disk() {
  USAGE=$(df -h / | awk 'NR==2{print $5}')
  echo "Disk usage on root: $USAGE"
}
check_disk
```

Make a Script & Run It

```
# Step 1: Create the script file
touch health-check.sh

# Step 2: Grant execute permission
chmod +x health-check.sh

# Step 3: Run it
```

```
./health-check.sh

# Or run it with bash explicitly
bash health-check.sh
```

Automation DevOps: Scripts are used daily for: deployment pipelines, log rotation, server health checks, backup jobs, cluster node setup, and notification alerts. A good DevOps engineer automates everything repetitive.

11. Linux to Kubernetes Connection Map

Everything in Kubernetes is built on Linux concepts. Mastering Linux makes Kubernetes 10x easier to learn and debug. Here is how they map to each other.

Easy Analogy: Linux and Kubernetes are like building a single city (Linux) vs. managing thousands of cities at national scale (Kubernetes). The same roads, electricity, and water systems exist -- they're just automated, replicated, and self-healing across hundreds of nodes.

Linux Concept	Kubernetes Equivalent	Real-World Use
`Process`	Container	App running inside a Pod
`Virtual Machine`	Node	Worker machine in cluster
`Daemon/Service`	Pod	Smallest deployable unit
`File System`	Volume	Persistent storage for Pods
`/var/log Logs`	kubectl logs	Debugging crashed Pods
`Port (netstat)`	Service Port	How Pods talk to each other
`User (useradd)`	ServiceAccount	Identity for pods in cluster

Full Pod Crash Debug Workflow

```
# Scenario: Pod is stuck in CrashLoopBackOff

# Step 1: Check pod status (like ps -ef on Linux)
kubectl get pods -n mynamespace

# Step 2: Read crash logs (like cat /var/log/app.log)
kubectl logs myapp-pod --previous

# Step 3: Full pod details (like systemctl status on Linux)
kubectl describe pod myapp-pod

# Step 4: Shell into the pod (like SSH into a server)
kubectl exec -it myapp-pod -- /bin/bash

# Step 5: Check disk & memory inside pod
df -h && free -m

# Step 6: Check if the app port is listening
ss -tulnp

# Step 7: Test internal connectivity
curl http://myservice.default.svc.cluster.local/health
```

Linux Mastery = DevOps Superpower

Files | Permissions | Processes | Networking | Scripting

Master Linux -> Master Kubernetes -> Master DevOps

@tech__.talks